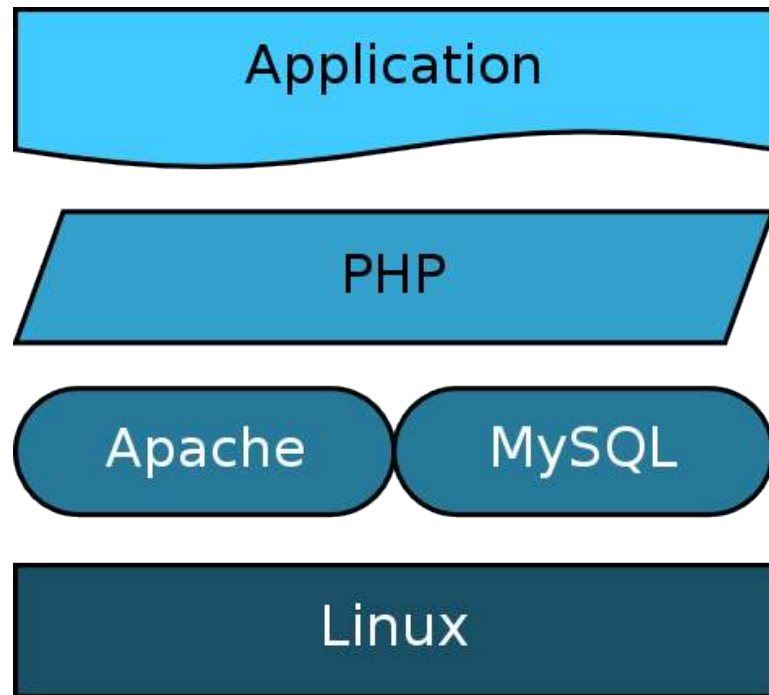




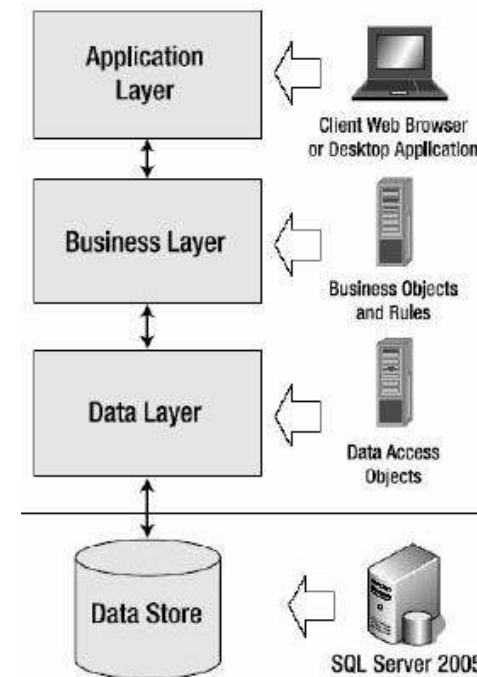
Phillip Trelford, @ptrelford

Code Mesh London 2015, #codemesh

LAMP Stack (State in DB)



3-Tier (State in memory & DB)





[Greencodds Tenth Rule Of Programming](#)

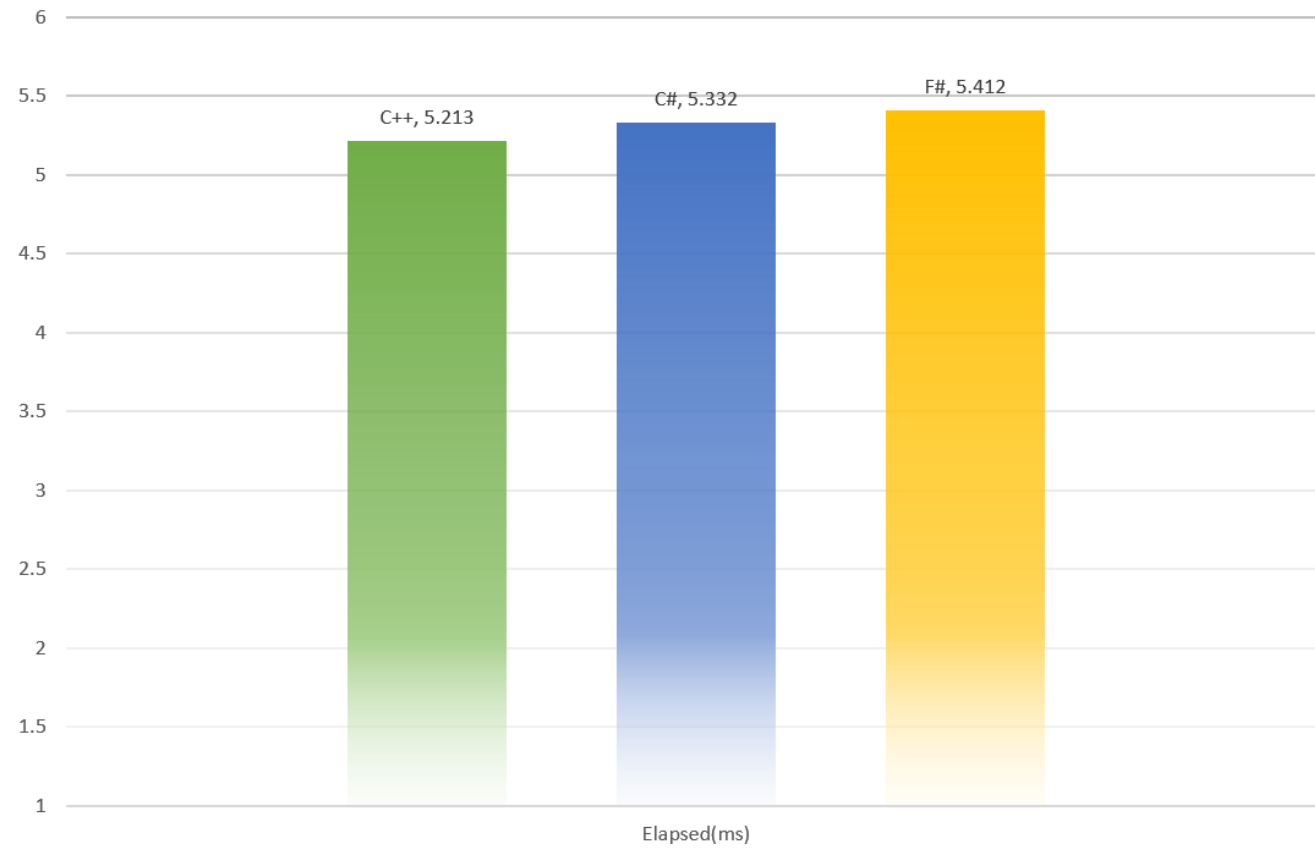
1 CPU cycle	0.3 ns	1 s
Level 1 cache access	0.9 ns	3 s
Level 2 cache access	2.8 ns	9 s
Level 3 cache access	12.9 ns	43 s
Main memory access	120 ns	6 min
Solid-state disk I/O	50-150 μ s	2-6 days
Rotational disk I/O	1-10 ms	1-12 months
Internet: SF to NYC	40 ms	4 years
Internet: SF to UK	81 ms	8 years
Internet: SF to Australia	183 ms	19 years
OS virtualization reboot	4 s	423 years
SCSI command time-out	30 s	3000 years
Hardware virtualization reboot	40 s	4000 years
Physical system reboot	5 m	32 millenia

Source: <http://blog.codinghorror.com/the-infinite-space-between-words/>

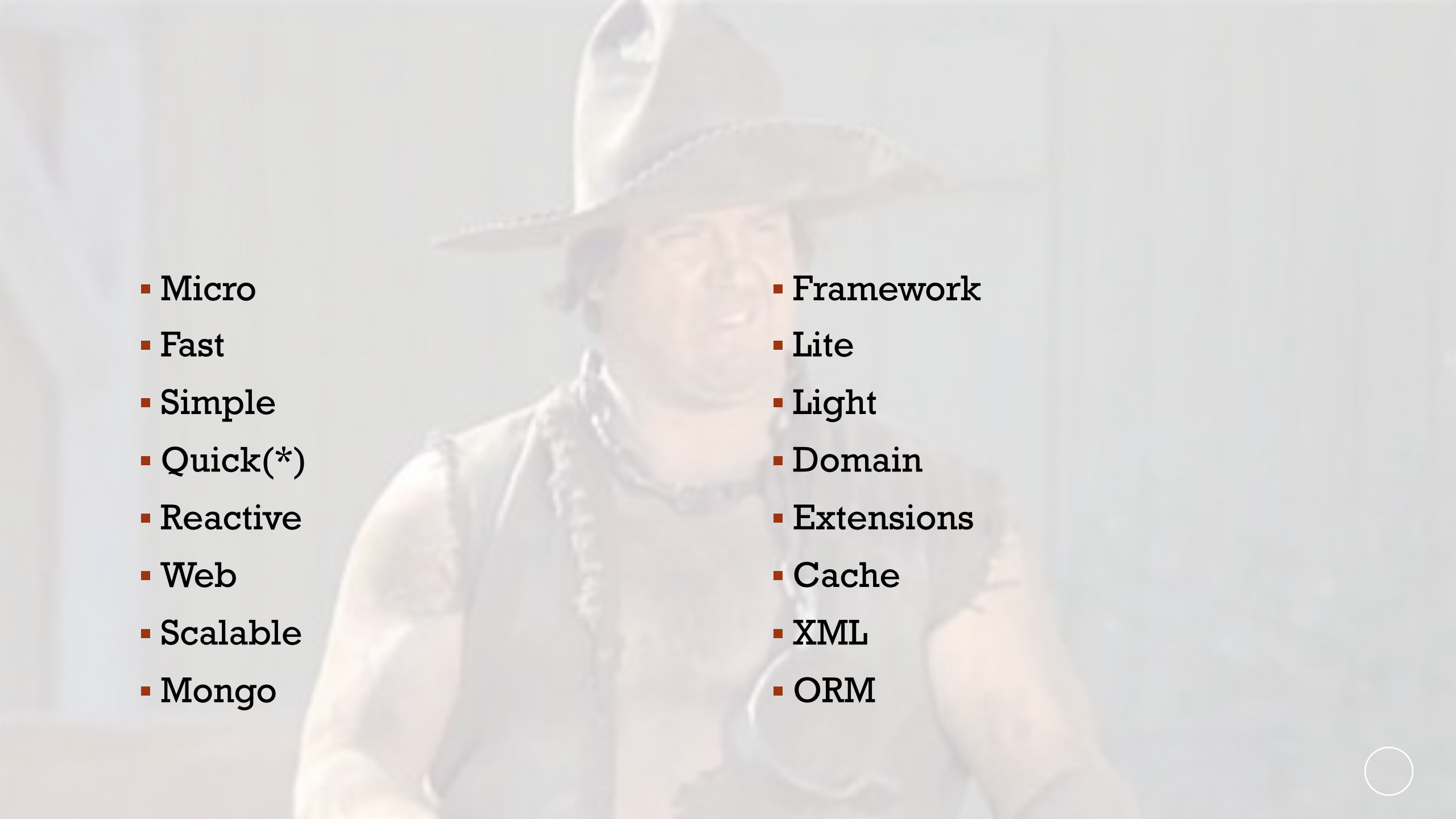
[**Google** found that] the page with 10 results took 0.4 seconds to generate. The page with 30 results took 0.9 seconds. **Half a second delay caused a 20% drop in traffic.** Half a second delay killed user satisfaction.

In A/B tests, [**Amazon**] tried delaying the page in increments of 100 milliseconds and found that **even very small delays would result in substantial and costly drops in revenue.**

Source: <http://blog.codinghorror.com/performance-is-a-feature/>



Source: <http://trelford.com/blog/post/C2b2b-vs-C-vs-F-vs-Haskell.aspx>

- 
- Micro
 - Fast
 - Simple
 - Quick(*)
 - Reactive
 - Web
 - Scalable
 - Mongo

- Framework
- Lite
- Light
- Domain
- Extensions
- Cache
- XML
- ORM



Phillip Trelford, @ptrelford

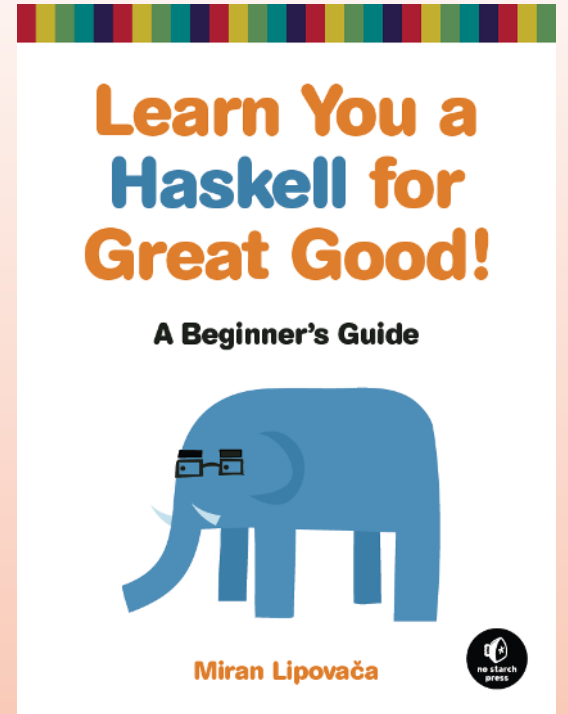
Code Mesh London 2015, #codemesh

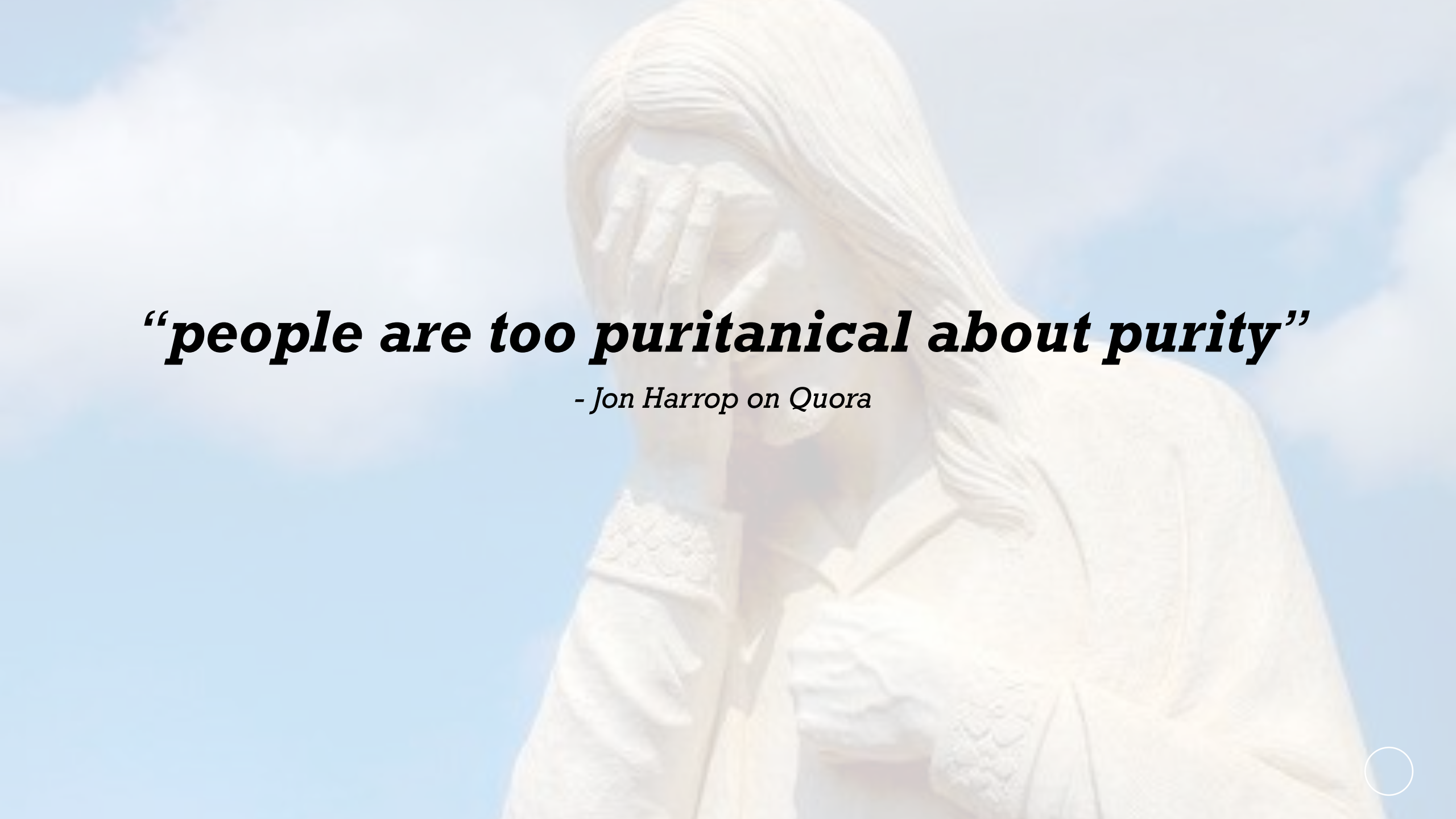

```
type 'a list =  
  | Empty  
  | Node of head:'a * tail:'a list
```



```
quicksort :: (Ord a) => [a] -> [a]
quicksort [] = []
quicksort (x:xs) =
    let smallerSorted = quicksort [a | a <- xs, a <= x]
        biggerSorted = quicksort [a | a <- xs, a > x]
    in smallerSorted ++ [x] ++ biggerSorted
```

* well it's short and a sort but it's not quick!





“people are too puritanical about purity”

- Jon Harrop on Quora



Phillip Trelford, @ptrelford

Code Mesh London 2015, #codemesh



```
var assigned = new Dictionary<Developer, int>();
```

```
foreach (var developer in developers)
```

▼

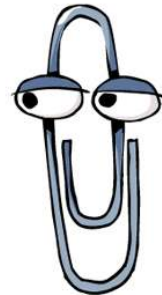
 Convert into LINQ-expression

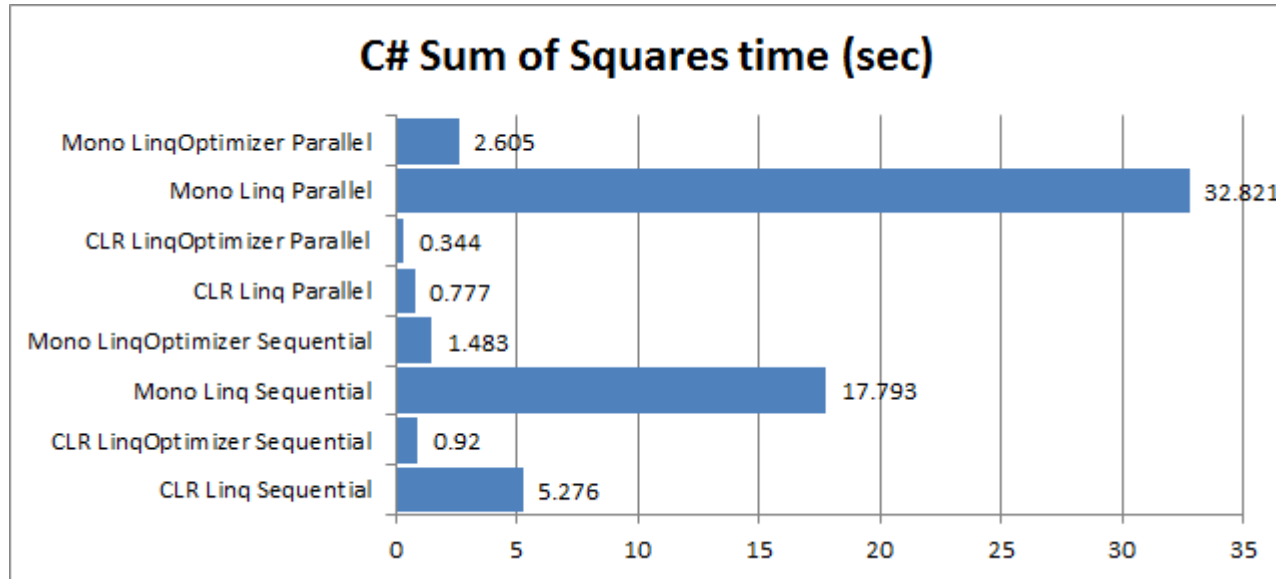
 Convert to for

 Inspection options for "Loop can be converted into LINQ-expression"

 Suppress inspection "Loop can be converted into LINQ-expression" with comment

ignedIssues);





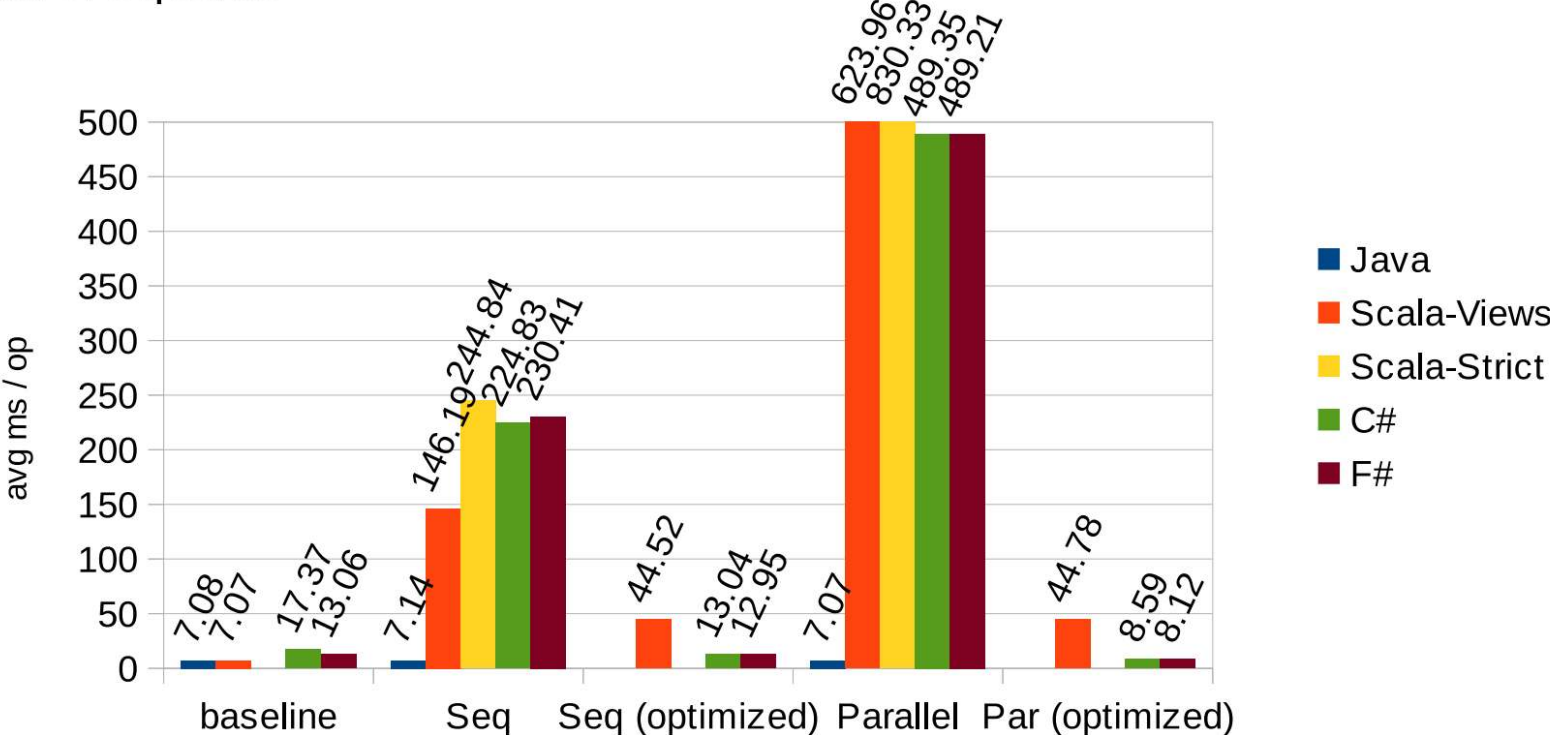
LinqOptimizer compiles declarative LINQ queries into fast loop-based imperative code. The compiled code has fewer virtual calls and heap allocations, better data locality and **speedups of up to 15x**



Phillip Trelford, @ptrelford

Code Mesh London 2015, #codemesh

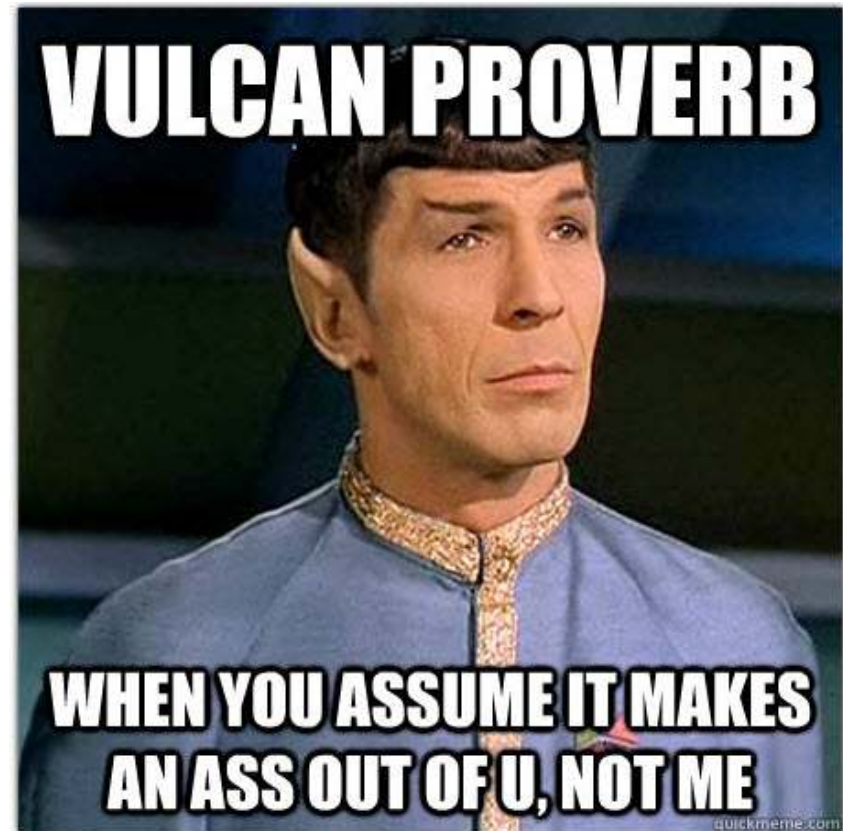
Sum of Squares



@sforkmann @ptrelford @dsyme how
efficient memory and perf wise to
@ReactiveX? Our numbers are pretty hard
to beat!

Reactive Extensions (C#)	F# Observable module	Nessos Streams
<pre> let rxValue = data .ToObservable() .Where(fun x -> x%2L = 0L) .Select(fun x -> x * x) .Sum() .ToEnumerable() > Seq.head // Real: 00:00:02.895, // CPU: 00:00:02.843, // GC gen0: 120, gen1: 0, gen2: 0 </pre>	<pre> let obsValue = data > Observable.ofSeq > Observable.filter (fun x -> x%2L = 0L) > Observable.map (fun x -> x * x) > Observable.sum > Observable.first // Real: 00:00:00.479, // CPU: 00:00:00.468, // GC gen0: 18, gen1: 0, gen2: 0 </pre>	<pre> let streamValue = data > Stream.ofArray > Stream.filter (fun x -> x%2L = 0L) > Stream.map (fun x -> x * x) > Stream.sum // Real: 00:00:00.130, // CPU: 00:00:00.109, // GC gen0: 0, gen1: 0, gen2: 0 </pre>

- Be scientific
 - Do test multiple implementations
 - Don't set out to confirm your bias
 - Instrument and profile your code



Phillip Trelford, @ptrelford

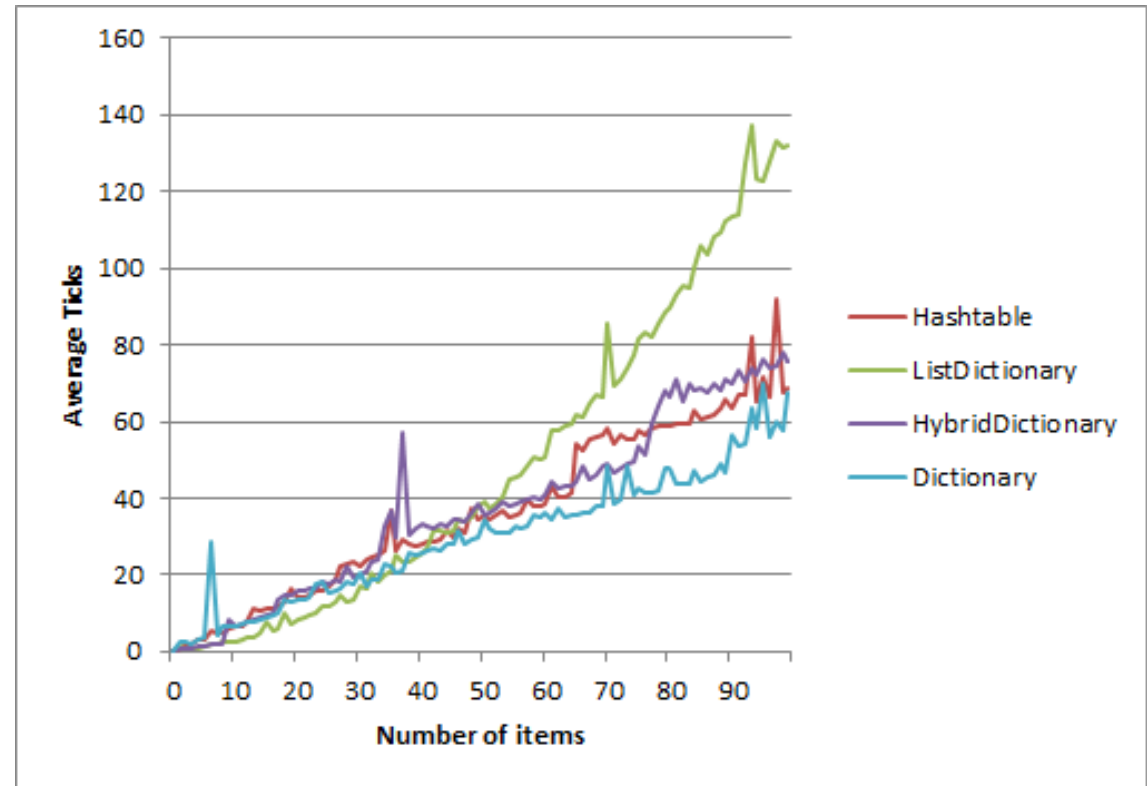
Code Mesh London 2015, #codemesh

	Adding 1M items	Slower than best	Iterating 1M items	Slower than best
SortedDictionary	0.7713	59.3333x	0.0873	4.5172x
Dictionary (with initial capacity)	0.0423	3.2564x	-	-
Dictionary	0.0810	6.2308x	0.0350	1.8103x
Map (using ofList)	3.1750	244.2308x	-	-
Map	3.1537	242.5897x	0.1253	6.4828x
Array (from empty)	0.0987	7.5897x	-	-
Array	0.0130	1x	0.0193	1x

Source: <http://theburningmonk.com/benchmarks/>

HybridDictionary attempts to optimize Hashtable.

It implements a linked list and hash table data structure, switching over to the second from the first when the number of elements increases past a certain threshold.



Row Oriented Database

<u>date</u>	<u>price</u>	<u>size</u>
2011-01-20	10.1	10
2011-01-21	10.3	20
2011-01-22	10.5	40
2011-01-23	10.4	5
2011-01-24	11.2	55
2011-01-25	11.4	66
...	...	...
2013-03-31	17.3	100

Table of Data

<u>date</u>	<u>price</u>	<u>size</u>
2011-01-20	10.1	10
2011-01-21	10.3	20
2011-01-22	10.5	40
2011-01-23	10.4	5
2011-01-24	11.2	55
2011-01-25	11.4	66
...	...	...
2013-03-31	17.3	100

Column Oriented Database

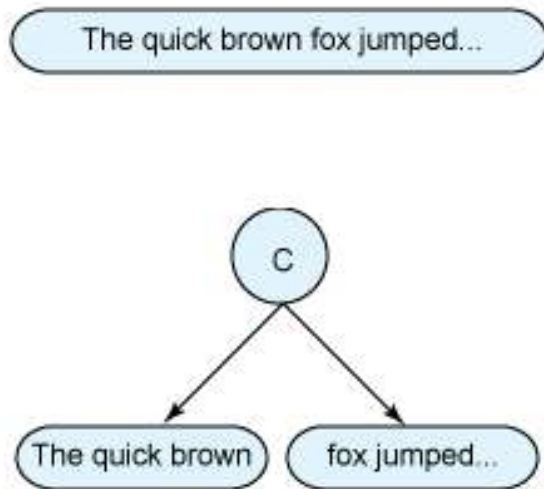
<u>date</u>	<u>price</u>	<u>size</u>
2011-01-20	10.1	10
2011-01-21	10.3	20
2011-01-22	10.5	40
2011-01-23	10.4	5
2011-01-24	11.2	55
2011-01-25	11.4	66
...	...	...
2013-03-31	17.3	100

Source: <http://www.timestored.com/kdb-guides/kdb-database-intro>

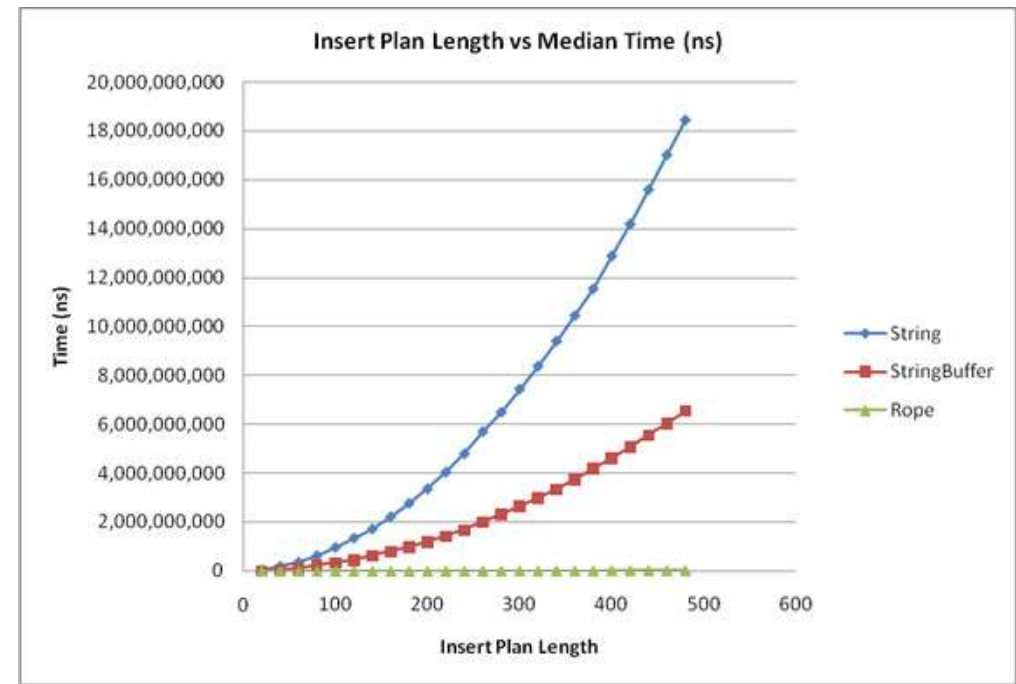
Phillip Trelford, @ptrelford

Code Mesh London 2015, #codemesh

String representation

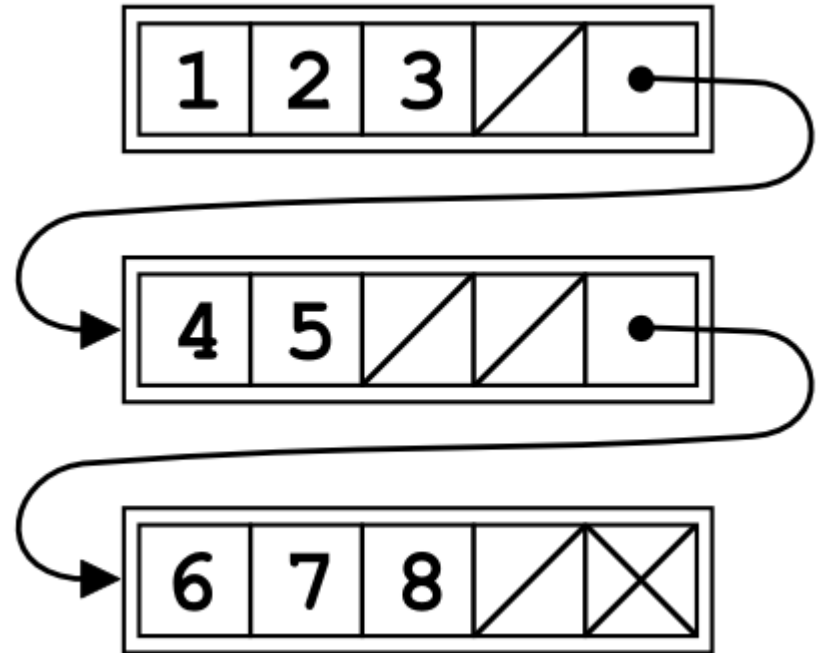


Insert Plan Length vs Median Time

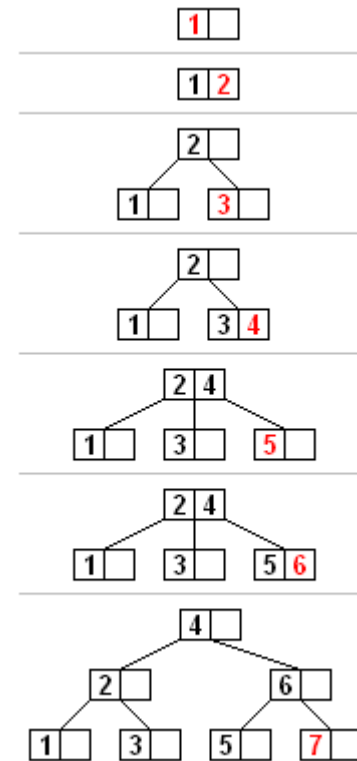
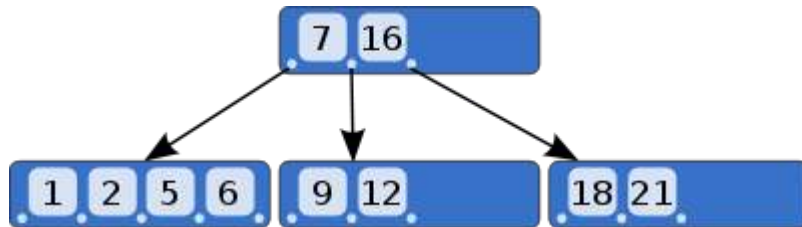


Source: <http://www.ibm.com/developerworks/library/j-ropes/>

- A linked-list of arrays



- A tree of arrays



[the B+ tree] is one of the most beautiful and useful inventions of computer science, with significance to civilization rivalling the invention of the arch, double entry accounting, and arabic numerals

Source: <http://bplusdotnet.sourceforge.net/>

Phillip Trelford, @ptrelford

Code Mesh London 2015, #codemesh

- Prefer safe and simple defaults - immutability
- Assume nothing – profile everything
- Mutable arrays are fast
- Build fast data structures from arrays
- Be pragmatic and they'll be your friend

fsharpWorks

- Twitter
 - @ptrelford
- Blog
 - <http://trelford.com/blog>
- Unrolled linked list in OCaml
 - <http://github.com/ptrelford/Unrolled>

